Shuttle Dispatch

Value and Policy Iteration

The infinite-horizon solution is to **wait when fewer than 13 passengers are waiting and dispatch when 13 or more are waiting**. The sections below specify the model, explain each solution method, and present numerical evidence that the implementation is correct.

1 The model: what happens in one period?

Each period has four steps: observe the queue, choose whether to dispatch, charge the dispatch and waiting costs, and then add the new arrivals. The state records only the queue length because the assumed arrival distribution and available actions do not otherwise depend on the past.

Table 1: Notation used to describe the environment.

Symbol	Meaning
t	Decision period; the first decision is at $t = 0$.
s_t	Number of passengers waiting *before* the decision at time t. Possible states are $0, 1, \ldots, 200$.
a_t	Action: 0 means wait; 1 means dispatch one shuttle.
N_t	Random number of new arrivals after the decision. Each of $1, 2, 3, 4, 5$ has probability $1/5$. Arrivals are independent across periods.
$c(s, a)$	Cost of taking action a when the queue contains s passengers.
γ	Discount factor, 0.95. A cost one period later receives weight 0.95 relative to a cost now.

A shuttle can serve 15 passengers. Therefore, after action a in state s, the number still waiting is $\max\{s - 15a, 0\}$. The maximum with zero prevents the queue from becoming negative. The cost is

$$\underbrace{c(s,a)}_{\text{current cost}} = \underbrace{100a}_{\text{dispatch cost}} + \underbrace{2\max\{s-15a,0\}}_{\text{cost of passengers left waiting}} . \tag{1}$$

New arrivals are added after this cost is assessed. The next queue is

$$s_{t+1} = \min\{\max\{s_t - 15a_t, 0\} + N_t,\ 200\}\,. \tag{2}$$

The minimum with 200 enforces the station capacity.

Example. With 20 passengers, waiting costs $2(20) = 40$ and leaves 20 passengers before arrivals. Dispatching serves 15, leaves 5, and costs $100 + 2(5) = 110$. If three passengers then arrive, the next queue is 23 after waiting or 8 after dispatch. Dispatching costs more *now*, but may save enough future waiting cost to be optimal.

Additional assumptions. Only one shuttle can depart per period, with no separate fleet or travel-time restriction. New arrivals first incur waiting cost at the next decision. Overflow above 200 is discarded without an extra penalty. These timing and overflow choices are assumptions, rather than consequences of the queue cap alone. Ties between actions are resolved by waiting.

2 Finite-horizon enumeration: 500 decisions

2.1 The quantity being optimized

Let $J_t(s)$ be the **minimum expected discounted cost still to be paid**, starting with s passengers at time t. Costs are discounted relative to that time. With 500 decisions, its definition is

$$J_t(s) = \min_{\pi} \mathbb{E}_{\pi}\left[\sum_{u=t}^{499} \gamma^{u-t} c(s_u, a_u) \,\middle|\, s_t = s\right]. \tag{3}$$

Here u counts future periods; π denotes a policy, meaning a rule for choosing actions; and $\mathbb{E}_{\pi}$ means an average over arrival randomness when that policy is followed. A smaller $J_t(s)$ is better. The report uses *cost values*; if rewards were defined as negative costs, their values would have the opposite sign.

2.2 Backward induction over states and actions

Part (a)'s "enumeration" is interpreted as evaluating every state, both actions, and all five possible arrivals at each period. There is no terminal penalty, so $J_{500}(s) = 0$. Starting at the last decision and working backward, compute

$$Q_t(s,a) = c(s,a) + \frac{\gamma}{5}\sum_{k=1}^{5} J_{t+1}(\min\{\max\{s-15a, 0\} + k, 200\}), \tag{4}$$

$$J_t(s) = \min_{a\in\{0,1\}} Q_t(s,a). \tag{5}$$

$Q_t(s,a)$ is the expected remaining cost if action a is taken *now* and decisions are optimal afterward. In the sum, k is a possible arrival count. The factor $1/5$ averages the five outcomes, and γ discounts their continuation costs. The minimizing action is saved as $\pi_t(s)$, the decision rule at time t.

This procedure reuses previously computed continuation values, avoiding the exponential cost of listing every complete decision history. The implementation saves all 501 value vectors and all 500 policy vectors.

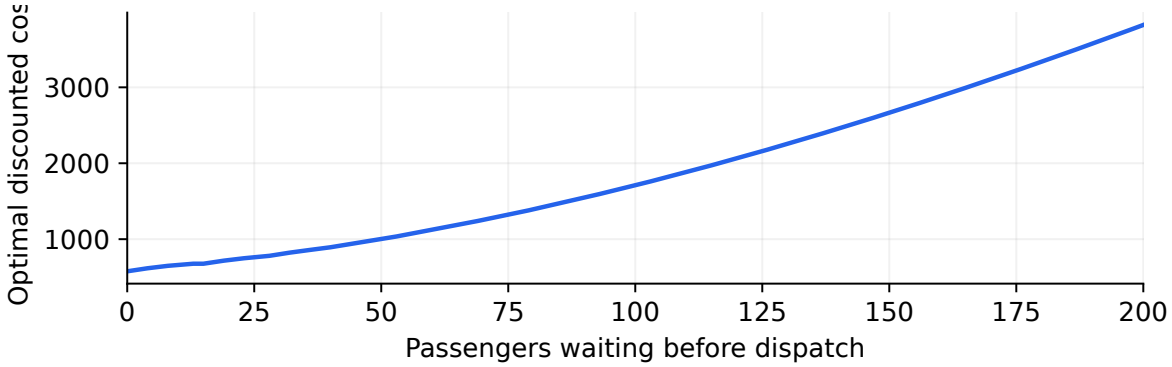


Figure 1: Minimum expected discounted cost at the initial decision, $J_0(s)$, for 500 decisions and zero terminal cost.

The finite-horizon policy need not be stationary: it can change as the end approaches. In the final period, waiting is optimal in every state. A dispatch costs 100 but saves at most $2 \times 15 = 30$ immediately; there is no later benefit or terminal penalty to justify it.

3 Infinite-horizon value iteration

For an unlimited sequence of decisions, write $J^*(s)$ for the minimum expected discounted cost starting from queue s. The superscript $*$ indicates optimality. This value has no time subscript because the problem has no deadline and the environment does not change over time.

3.1 The Bellman update

Starting from $J^{(0)}(s) = 0$, repeatedly compute

$$J^{(n+1)}(s) = \min_{a\in\{0,1\}} \left[c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J^{(n)}(\min\{\max\{s-15a,0\}+k, 200\}) \right]. \tag{6}$$

The superscript (n) counts algorithm iterations, not decision periods. Each update asks: *which action has the lowest current cost plus discounted expected future cost?* Its structure is the same as the finite-horizon recursion, but it improves an estimate of the stationary value instead of moving backward through time.

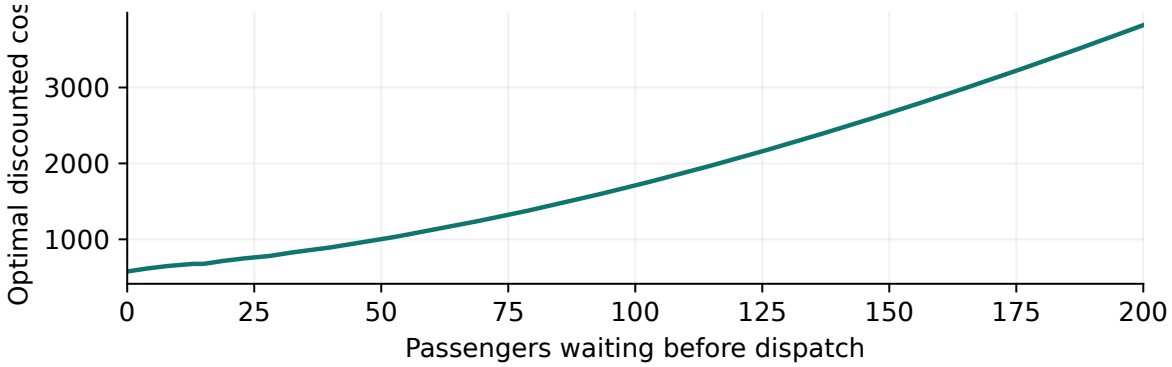


Figure 2: Optimal infinite-horizon cost $J^*(s)$, estimated by value iteration. Larger initial queues incur greater total expected cost.

3.2 When does iteration stop?

Let $\mathcal{T}J$ denote the entire right-hand side of Equation (6), evaluated using a candidate value vector J. This is the *Bellman operator*: one application performs one value update. Define its residual as

$$\delta(J) = \max_{s\in\{0,\ldots,200\}} |(\mathcal{T}J)(s) - J(s)|. \tag{7}$$

The residual measures how much another Bellman update would change the candidate values, in the worst state. The algorithm stops when $\delta(J) \le 10^{-10}$, which required **518 updates**.

Because $\gamma < 1$, the Bellman operator contracts maximum errors by at most γ. Consequently,

$$\max_s |J(s) - J^*(s)| \le \frac{\delta(J)}{1-\gamma}. \tag{8}$$

This connects a computable stopping criterion to the accuracy of the value estimate.

The implementation minimizes costs. Thus, its arrays named `finite_costs`, `vi_cost`, and `pi_cost` are the cost values J_t and J^* described above.

4 Policy iteration and the dispatch rule

Policy iteration alternates between evaluating a fixed decision rule and improving it. It starts from the rule "always wait."

Policy evaluation. For a stationary policy π, $\pi(s)$ is its action in state s. Let J_π be the vector of expected discounted costs when that rule is followed indefinitely. Let P_π be the 201×201 transition matrix: entry (s, s') is the probability of next queue s' after taking action $\pi(s)$. Let c_π be the vector with entries $c(s, \pi(s))$. Then

$$J_\pi = c_\pi + \gamma P_\pi J_\pi, \qquad \text{so} \qquad (I - \gamma P_\pi) J_\pi = c_\pi. \tag{9}$$

Here I is the identity matrix, and s' denotes a possible next state. The linear system computes the cost of the current policy. When several arrival counts produce the same next state, their probabilities are added in P_π.

Policy improvement. For every state, compare both actions using J_π as the continuation value in Equation (6). Replace the current action with the lower-cost one. Stop when no actions change. This took **six evaluation/improvement rounds**, including the final stability check.

The resulting optimal stationary rule is

$$\boxed{\pi^*(s) = \begin{cases} 0 & \text{(wait)}, \quad 0 \le s \le 12, \\ 1 & \text{(dispatch)}, \quad 13 \le s \le 200. \end{cases}} \tag{10}$$

The threshold was discovered by comparing both actions in every state; it was not imposed on the search. Departing with 13 or 14 passengers can be worthwhile because the savings in future holding costs outweigh the benefit of waiting for a full shuttle.

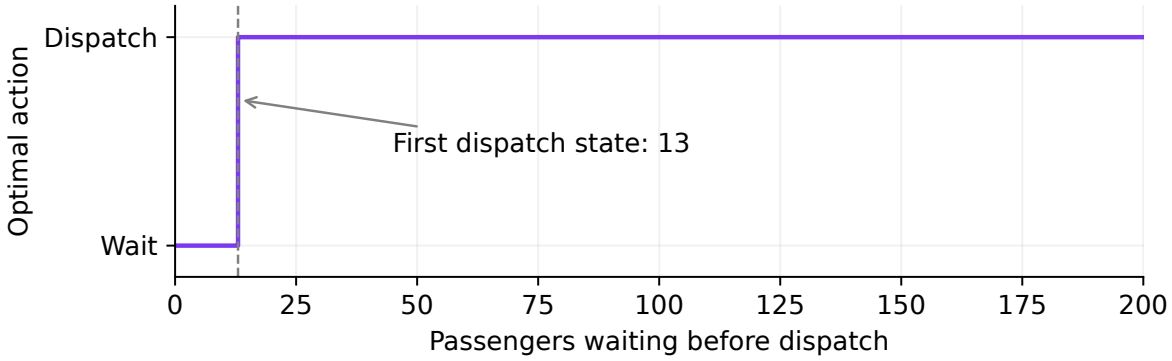


Figure 3: Optimal stationary policy from policy iteration. The first dispatch state is 13.

Table 2: Representative minimum infinite-horizon discounted costs.

Initial passengers s	Optimal cost $J^*(s)$
0	576.670168
15	676.670168
200	3821.586477

The difference $J^*(15) - J^*(0) = 100$ is a useful consistency check. Dispatch at 15 clears the queue, while waiting at zero also leaves it empty. Both lead to the same next-arrival distribution; only the dispatch charge differs.

5 Edge cases and correctness checks

All **14 validation checks passed**. The checks use optimality conditions, independent solution procedures, and model changes with predictable outcomes.

5.1 Agreement and optimality

Value iteration and policy iteration selected identical actions in all 201 states. Their maximum value difference was 1.964×10^{-9}. The policy-iteration result had Bellman residual 9.095×10^{-13}. Equation (8) therefore bounds its maximum value error by approximately 1.819×10^{-11}, subject to floating-point rounding. The smallest absolute difference between the two action costs was 3.778, well above the numerical error.

The maximum difference between the 500-period time-zero values and infinite-horizon values was 4.942×10^{-9}. Since the largest possible stage cost is 470, a conservative bound on the discounted cost beyond 500 periods is

$$\sum_{t=500}^{\infty} \gamma^t \, 470 = \frac{470\gamma^{500}}{1-\gamma} = 6.838 \times 10^{-8}. \tag{11}$$

The observed difference is smaller than this bound. Agreement is expected because very late costs receive little weight under discounting.

5.2 Queue boundaries and the final period

Service never produces a negative queue: 15 passengers clear exactly, and 16 leave one. At state 200, waiting always gives next state 200; dispatch gives next states 186 through 190. At zero, dispatch costs 100 more than waiting without changing the next-state distribution, so it is strictly dominated. These cases verify that service, saturation, and probability pooling are implemented consistently.

The terminal value vector was checked to be zero. The last-period policy was also checked against the analytic prediction that waiting is optimal everywhere. This tests that the finite-horizon algorithm does not charge continuation costs beyond its deadline.

Table 3: Model modifications with known answers. In this table, $J(s)$ denotes the optimal cost for the modified problem.

Modification	Predicted result	Observed result
Zero holding cost	Always wait; $J(s) = 0$.	Matched all states.
Zero shuttle capacity and no arrivals	Always wait; $J(s) = 2s/(1-\gamma)$.	Zero analytic error.
Free dispatch, capacity 200, no arrivals	Dispatch at nonempty states; $J(s) = 0$.	Matched all states.
Zero discount factor	Only current costs matter: always wait, with $J(s) = 2s$.	Matched all states.

A separately coded exhaustive decision/arrival tree for a seven-state, four-period example matched backward induction exactly. Complete enumeration is feasible at this scale and provides an independent check of the finite-horizon recursion.

6 Independent simulation and reproducibility

The optimal policy was simulated from initial queues of 0, 15 and 200. Each starting state used 20,000 independent trajectories of 300 decisions, with random seed 1668. A trajectory's total cost is the sum of its stage costs weighted by $1, \gamma, \gamma^2, \ldots$. The simulator calculated transitions and costs directly instead of reusing the solver's transition-array builder.

Table 4: Simulation estimates compared with exact policy evaluation.

Initial queue	Simulation mean	Exact cost	Standard error
0	576.671469	576.670168	0.219213
15	676.814669	676.670168	0.221520
200	3821.605190	3821.586477	0.532280

The standard error estimates uncertainty in the simulation *mean*: it is the sample standard deviation of trajectory costs divided by $\sqrt{20{,}000}$. Each mean was within four standard errors of the exact value, allowing an additional 0.001951 for the conservative bound on costs omitted after period 300. This wider tolerance is the automated simulation check; Figure 4 displays approximate 95% intervals using ± 1.96 standard errors.

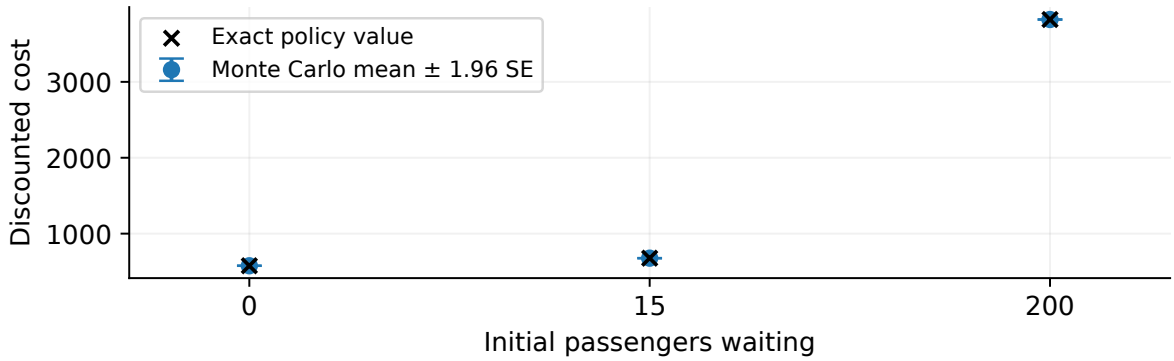


Figure 4: Independent simulation means and exact policy costs. Error bars describe uncertainty in the mean, not the variation of individual trajectory costs.

Simulation supports the computed policy values but does not prove optimality. The Bellman residual provides the stronger optimality check. Likewise, algorithm agreement alone cannot catch every modeling mistake because the methods share the same specification; the analytic examples and independently coded tree and simulator provide additional evidence.

6.1 Reproducing the results

From the project folder, run:

```
python -m pip install -r requirements.txt
python shuttle.py
jupyter lab shuttle_plots.ipynb
```

The solver runs all three methods and all validation checks, raising an error if any check fails. Full evidence is saved in `results/summary.json`; per-state results are in `results/state_results.csv`. The saved arrays in `results/shuttle_results.npz` include every finite-horizon value and policy. Running all notebook cells regenerates the assignment plots in `results/figures/`. For the compact vector figures used in this report, run `make_report_figures.py`; then compile `SHUTTLE_REPORT.tex`.

7 Validation with an additional three months

An extended study would focus on three questions.

How sensitive is the dispatch rule to the parameters?

Vary dispatch and holding costs, shuttle capacity, discounting, and arrival distributions. Record changes in the optimal threshold and test whether threshold behavior persists. Compare larger queue caps, such as 250, 500 and 1,000, and introduce penalties for lost arrivals. These experiments would reveal whether the 13-passenger recommendation depends strongly on the chosen parameters or overflow assumption.

Can another method confirm the solution?

Construct an independent linear-programming formulation and compare its values and actions with dynamic programming. Use higher precision when action costs are nearly tied. Vary the finite horizon and terminal penalties to study decisions near the deadline. Use more simulation seeds and starting queues, and measure how often nominal confidence intervals contain the exact policy value.

Does the policy remain effective under more realistic conditions?

Replace independent uniform demand with bursty or correlated arrivals, and evaluate the original policy under these alternative demand processes. Add shuttle availability and travel times to the state. These extensions would test robustness to modeling assumptions, beyond verifying numerical correctness for the stated environment.